EXPERIMENT 1

Shravanya Andhale

D20A-02

Getting Started with Python, TensorFlow and Hugging Face Transformers

AIM:

To install and configure the Generative AI development environment using Google Colab, Python, TensorFlow/PyTorch, and the Hugging Face Transformers library, and execute a pre-trained transformer model to generate text from user prompts.

THEORY:

Generative Artificial Intelligence (Generative AI) is a branch of Artificial Intelligence that focuses on developing intelligent systems capable of creating new and original content. Unlike traditional AI systems, which are mainly designed to classify, predict, recognize patterns, or make decisions based on existing data, generative AI models learn the underlying patterns and structures present in large datasets and use this knowledge to generate new outputs. These outputs may include text, images, audio, video, computer code, music, and other forms of digital content.

Generative AI has become one of the most important areas of modern artificial intelligence because of its wide range of practical applications. It is used in applications such as conversational chatbots, automated content generation, language translation, text summarization, image creation, speech synthesis, programming assistance, recommendation systems, and virtual assistants. The development of powerful deep learning models and the availability of large amounts of training data have significantly contributed to the rapid growth of generative AI.

Python for Artificial Intelligence

Python is one of the most widely used programming languages in the fields of Artificial Intelligence, Machine Learning, and Deep Learning. Its popularity is mainly due to its simple and readable syntax, ease of learning, flexibility, and extensive ecosystem of scientific and AI-related libraries. Python allows researchers, students, and developers to implement complex algorithms with relatively simple code.

Several important Python libraries support AI and data science applications. Libraries such as NumPy and Pandas are commonly used for numerical computation and data manipulation, while libraries such as Matplotlib can be used for data visualization. For machine learning and deep learning applications, popular frameworks include TensorFlow and PyTorch. These frameworks provide tools for creating, training, testing, and deploying neural network models.

Because of its large community and extensive collection of open-source libraries, Python provides an efficient environment for experimenting with AI models and developing intelligent applications machine learning algorithms, and generative AI models without requiring high-performance local hardware.

Deep Learning Frameworks: TensorFlow and PyTorch

TensorFlow and PyTorch are two widely used deep learning frameworks that provide tools for building and working with neural networks. They allow developers to perform mathematical operations efficiently and support the training and execution of complex deep learning models.

TensorFlow provides a comprehensive ecosystem for developing machine learning and deep learning applications. It supports the creation and training of neural networks and can be used for applications such as image recognition, natural language processing, speech recognition, and predictive analytics.

PyTorch is another popular deep learning framework, particularly in research and experimentation. It provides a flexible programming environment and is widely used for developing and testing new neural network architectures. Both TensorFlow and PyTorch support hardware acceleration and are capable of handling the large-scale computations required by modern AI systems.

Hugging Face Transformers Library

The Hugging Face Transformers library is a widely used Python library that provides access to a large collection of pre-trained transformer-based models. These models have already been trained on large amounts of data and can be used directly or further adapted for specific tasks.

The library supports a variety of Natural Language Processing tasks, including:

- Text generation
- Text classification
- Language translation
- Text summarization
- Question answering
- Sentiment analysis
- Conversational AI
- Token classification

One of the major advantages of using pre-trained models is that users do not need to train a large neural network from the beginning. Training large transformer models requires significant amounts of data, computational resources, and time. Pre-trained models allow users to access powerful AI capabilities with relatively simple Python programs.

Transformer Models and GPT-2

Transformer-based models represent an important advancement in the field of Natural Language Processing. These models are designed to understand relationships between different words and tokens within a sequence of text. By learning patterns from large collections of text, transformer models can perform a variety of language-related tasks.

GPT-2 (Generative Pre-trained Transformer 2) is a transformer-based language model designed for text generation. The model is trained to predict the next word or token in a sequence based on the context provided by the previous text. By repeatedly predicting the next token, GPT-2 can generate complete sentences and paragraphs that are related to the input prompt.

For example, if a user provides the model with the beginning of a sentence or a short paragraph, the model analyzes the context and generates additional text that follows similar linguistic patterns. The quality and relevance of the generated output depend on factors such as the input prompt, model configuration, and generation parameters.

Text Generation Using the Pipeline

In this experiment, the GPT-2 model was accessed using the text-generation pipeline provided by the Hugging Face Transformers library. A pipeline provides a simplified interface that allows users to perform machine learning tasks without manually handling all the complex steps involved in loading the model and processing the input.

The general process of text generation involves the following steps:

1. Import the required Python libraries.
2. Load the pre-trained GPT-2 model using the text-generation pipeline.
3. Provide an input prompt or starting text.
4. The model analyzes the context of the prompt.
5. GPT-2 predicts possible subsequent tokens based on patterns learned during training.
6. The generated tokens are combined to produce additional text.
7. The final output is displayed to the user.

The model does not simply retrieve a previously stored answer. Instead, it generates new text by predicting probable sequences based on the relationships and patterns it learned from its training data. Because text generation involves probabilities, different outputs may be produced depending on the generation settings and parameters used.

CODE AND OUTPUT:

https://colab.research.google.com/drive/1GyzUAdX96GoXFEA12ZucEazPm65OpRiB?usp=sharing

```
import sys
print("Python version:", sys.version)
```

```
Python version: 3.12.13 (main, Mar  4 2026, 09:23:07) [GCC 11.4.0]
```

```
!pip install transformers torch tensorflow -q
```

```
from transformers import pipeline
```

```
generator = pipeline("text-generation", model="gpt2")
print("GPT-2 text generation pipeline loaded successfully.")
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
config.json: 100% 665/665 [00:00<00:00, 60.8kB/s]
model.safetensors: reconstructing file: 100% 548MB / 548MB, 44.3MB/s
model.safetensors: downloading bytes: 474MB, 35.8MB/s
Loading weights: 100% 148/148 [00:00<00:00, 1768.21it/s]
generation_config.json: 100% 124/124 [00:00<00:00, 2.29kB/s]
tokenizer_config.json: 100% 26.0/26.0 [00:00<00:00, 780B/s]
vocab.json: 100% 1.04M/1.04M [00:00<00:00, 11.4MB/s]
```

```
prompt = "Artificial Intelligence will shape the future of"

# Words/content to avoid
bad_words = ["violence", "war"]
bad_words_ids = [
    generator.tokenizer.encode(word, add_special_tokens=False)
    for word in bad_words
]

# Token IDs for beginning and ending constraints
bos_token_id = generator.tokenizer.encode(
    " Technology", add_special_tokens=False
)[0]

eos_token_id = generator.tokenizer.eos_token_id

result = generator(
    prompt,
    max_length=80,
    min_length=30,
    temperature=0.8,
    do_sample=True,
    top_k=50,
    top_p=0.95,
    repetition_penalty=1.2,
    no_repeat_ngram_size=2,
    bad_words_ids=bad_words_ids,
    forced_bos_token_id=bos_token_id,
    forced_eos_token_id=eos_token_id,
    num_return_sequences=1
)
```

```
[transformers] Passing `generation_config` together with generation-related arguments=({'do_sample', 'top_k', 'no_repeat_ngram_size', 'num_return_sequences', 'bad_words_ids', 'repetition_penalty', 'top_p', 'temperature', 'forced_eos_token_id
[transformers] Both `max_new_tokens` (=256) and `max_length`(=80) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_cl
[transformers] Ignoring clean_up_tokenization_spaces=True for BPE tokenizer GPT2Tokenizer. The clean_up_tokenization post-processing step is designed for WordPiece tokenizers and is destructive for BPE (it strips spaces before punctuation).
Prompt:
Artificial Intelligence will shape the future of

Generated Text:
Artificial Intelligence will shape the future of medicine by creating a technology that can help doctors, and other professionals in need through scientific advances.
"This is not just about healthcare," says Scott Jonsson, vice president of clinical research at AIB Research Group (AIBA). "It's also connected to our real world needs."

Parameters Used:
bad_words_ids      : Avoids specified words/content.
forced_bos_token_id : Forces a specific beginning token.
forced_eos_token_id : Forces an end-of-sequence token.
no_repeat_ngram_size: Prevents repeated word sequences.
```

```
--- Prompt 3 ---
Input: Sustainability can be improved through
Output: Sustainability can be improved through new technologies and technologies to reduce the burden of environmental degradation.

The United Nations Environment Programme (UNEP) promotes sustainable development through its Sustainable Development Goals. UNEP also works with more than 40 partner countries to promote sustainable development within their borders and pro

An emphasis on renewable energy sources is important for this strategy. The United States, Canada and other countries are leading the way in renewable energy generation.

The United Nations Environment Programme (UNEP) promotes sustainable development through its Sustainable Development Goals. UNEP also works with more than 40 partner countries to promote sustainable development within their borders and pro

The United States and other countries are leading the way in renewable energy generation. The United States, Canada and other countries are leading the way in renewable energy generation. The United Nations Framework
[transformers] Both `max_new_tokens` (=256) and `max_length`(=60) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_

--- Prompt 4 ---
Input: The future of healthcare will depend on
Output: The future of healthcare will depend on new strategies based on the best available evidence and a better understanding of the mechanisms behind the problem," said a statement from the American College of Cardiology.

Researchers from the Virginia General Hospital, a U.S. affiliate of Washington University in St. Louis, found that, when compared with their own patients, those who had heart surgery had higher per-capita heart rates. When compared with pa

"It's very clear that this is not the case with any other heart surgery. This is not something that we think is necessarily going to be a 'fixable' thing, but rather we think it's going to be found in the community, and that could be the f

The researchers analyzed data from a national survey of heart surgeons, using a standardized questionnaire.

They found that almost one-fifth of heart surgeons surveyed did not think they were at risk of dying from heart surgery in 2016, compared with two-thirds of U.S. heart surgeons.

Ransom

--- Prompt 5 ---
Input: Technology and human creativity together can
Output: Technology and human creativity together can overcome problems that cannot be solved by simple action. For example, consider the fact that as a matter of fact, the human race has been living in an incomplete and incomplete world fo

So if you want to understand how human progress could be made in the future, you must be able to understand that the human race has been living in an incomplete reality for a very long time, because of the human nature that makes it possib

It is no wonder that to have this knowledge we need to be able to go out and make the most of what we are able to give to the world. So we need to use the information we have to make decisions about how we will change the world.

The second point is that a human has only one level of consciousness, and that is the mental level. And this is only an

Terminal
```

```python
prompt = "Artificial Intelligence will shape the future of"

settings = [
    {"max_length": 50, "temperature": 0.5, "top_k": 30},
    {"max_length": 60, "temperature": 0.8, "top_k": 50},
    {"max_length": 80, "temperature": 1.0, "top_k": 70}
]

for i, setting in enumerate(settings, 1):
    result = generator(
        prompt,
        max_length=setting["max_length"],
        temperature=setting["temperature"],
        do_sample=True,
        top_k=setting["top_k"]
    )

    print(f"\n--- Experiment {i} ---")
    print("Parameters:", setting)
    print("Generated Text:")
    print(result[0]["generated_text"])
```

```
[transformers] Both `max_new_tokens` (=256) and `max_length`(=50) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation for more information.
[transformers] Both `max_new_tokens` (=256) and `max_length`(=60) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation for more information.

--- Experiment 1 ---
Parameters: {'max_length': 50, 'temperature': 0.5, 'top_k': 30}
Generated Text:
Artificial Intelligence will shape the future of the world.

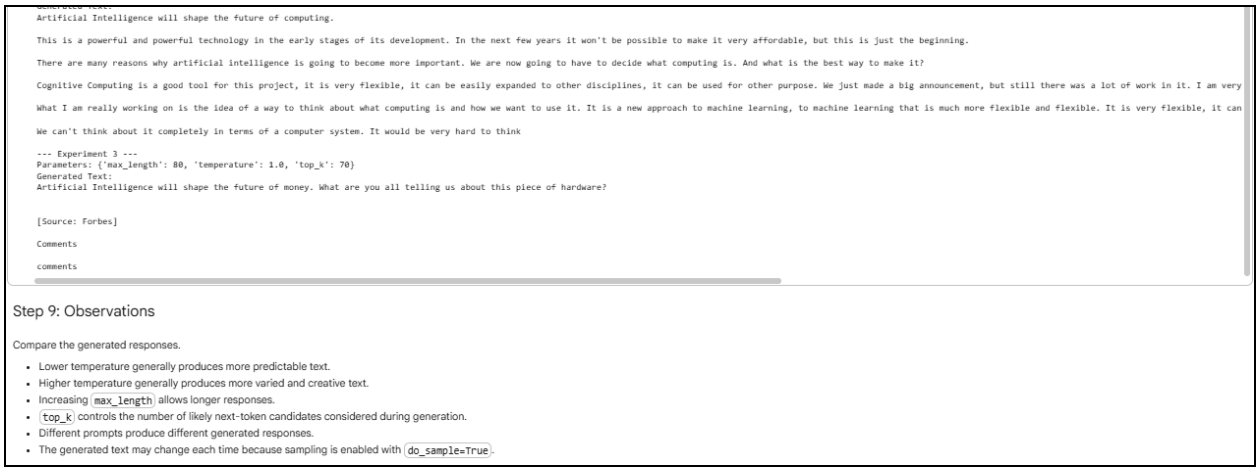
The Internet is changing the way we interact with technology, and we're beginning to see it in the form of machine learning.

The Internet is changing the way we interact with technology, and we're beginning to see it in the form of machine learning.

Machine learning is a new field. It's about understanding how to make things more human.

Machine learning is a new field. It's about understanding how to make things more human.

Machine learning is about understanding how to make things more human.
```




```
print("Model Used: GPT-2")
print("Task: Text Generation")
print("Library: Hugging Face Transformers")
print("Frameworks Installed: PyTorch and TensorFlow")
print("Prompts Tested: 5")
```

```
Model Used: GPT-2
Task: Text Generation
Library: Hugging Face Transformers
Frameworks Installed: PyTorch and TensorFlow
Prompts Tested: 5
```

RESULTS AND OBSERVATIONS:

The required libraries were installed successfully in Google Colab, and the Hugging Face text-generation pipeline was used to load the GPT-2 model. The model accepted a user prompt and generated a continuation of the input text. Different prompts produced different responses based on the context of each prompt.

The generation parameters influenced the output. A lower temperature generally produced more predictable responses, while a higher temperature produced more varied output. Increasing max_length allowed the model to generate longer text. The top_k parameter limited generation to a selected number of likely next-token candidates. Because do_sample was enabled, the generated response could vary between different executions of the same prompt.

The experiment demonstrated how a pre-trained transformer model can be used directly through a high-level pipeline without training a model from scratch. The generated results can be compared by changing prompts and generation parameters.

CONCLUSION:

The experiment successfully demonstrated the setup of a Generative AI development environment using Google Colab, Python, TensorFlow/PyTorch, and the Hugging Face Transformers library. A pre-trained GPT-2 transformer model was loaded and used to generate text from user prompts. The experiment also demonstrated the effect of parameters such as max_length, temperature, do_sample, and top_k on the generated output. Overall, the experiment provides a basic understanding of using pre-trained transformer models for text generation and forms a foundation for further Generative AI and Large Language Model applications.